

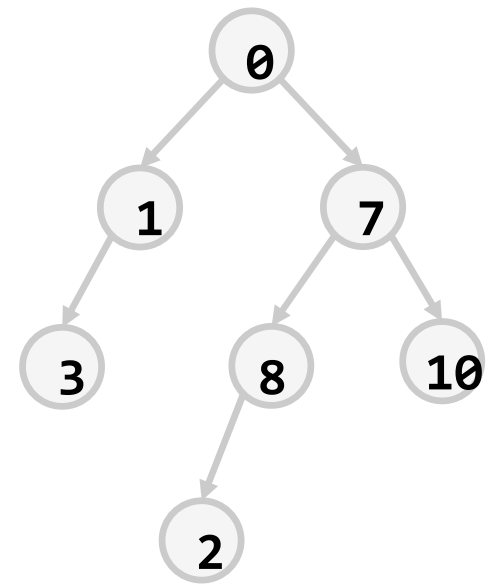
# Graphs and Applications

# Objectives (1 of 2)

- To model real-world problems using graphs and explain the Seven Bridges of Königsberg problem.
- To describe the graph terminologies: vertices, edges, simple graphs, weighted/unweighted graphs, and directed/undirected graphs.
- To represent vertices and edges using lists, edge arrays, edge objects, adjacency matrices, and adjacency lists.
- To model graphs using the **Graph** interface, the **AbstractGraph** class, and the **UnweightedGraph** class.
- To represent the traversal of a graph using the **UnweightedGraph.SearchTree** class.
- To design and implement depth-first search.

# Review Trees

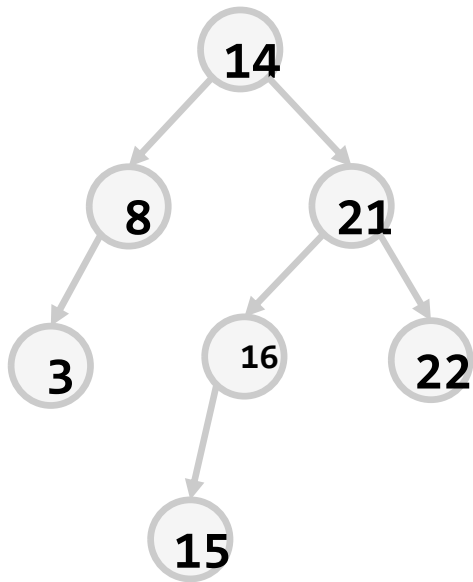
- A **tree** is a collection of nodes where each node has at most 1 parent and at least 0 children
  - A **binary tree** is a tree where each node has at most 2 children
- **Root node**: the single node with no parent, “top” of the tree
- **Leaf node**: a node with no children
- **Subtree**: a node and all its descendants
- **Edge**: connection between parent and a child



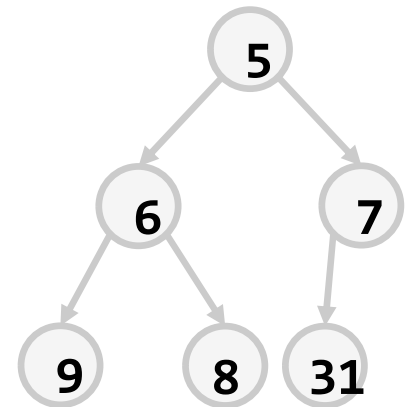
# *Review* Trees We've Seen So Far

## Binary Search Trees

- And variant: AVL Trees



## Binary Min-Heaps



# Inter-data Relationships

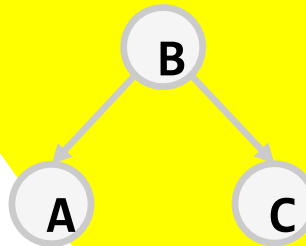
## Arrays

- Elements only store pure data, no connection info
- Only relationship between data is order



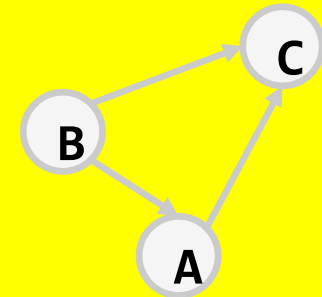
## Trees

- Elements store data and connection info
- Directional relationships between nodes; limited connections



## Graphs

- Elements AND connections can store data
- Relationships dictate structure; huge freedom with connections



# Everything is Graphs



- **Everything** is graphs.
- Most things we've studied so far can be represented by graphs.
  - BSTs are graphs
  - Linked lists? Graphs.
  - Heaps? Also can be represented as graphs.
  - Those trees we drew in the tree method? Graphs.
- But it's not just data structures that we've discussed...
  - Google Maps database? Graph.
  - Facebook? They have a "graph search" team. Because it's a graph
  - Those pictures of prerequisites in your program? Graphs.
  - Family tree? That's a graph

# Applications

## □ Physical Maps

- Airline maps
  - Vertices are airports, edges are flight paths
- Traffic
  - Vertices are addresses, edges are streets

## □ Relationships

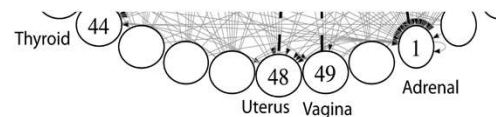
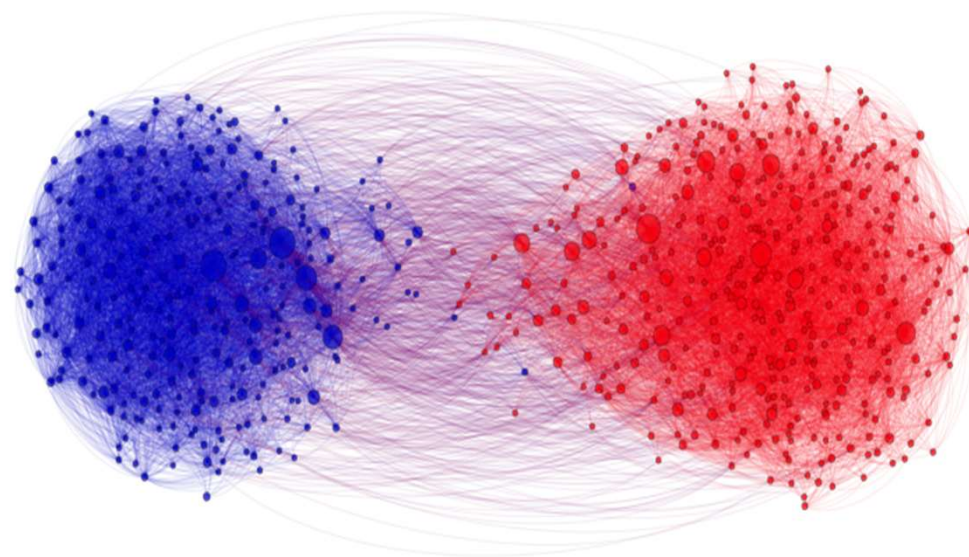
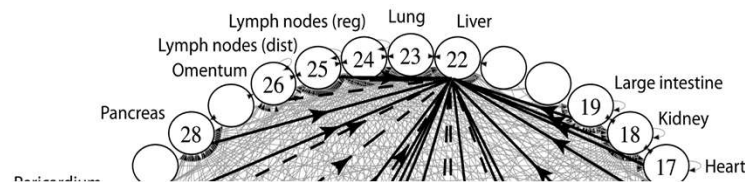
- Social media graphs
  - Vertices are accounts, edges are follower relationships

## □ Influence

- Biology
  - Vertices are cancer cell destinations, edges are migration paths

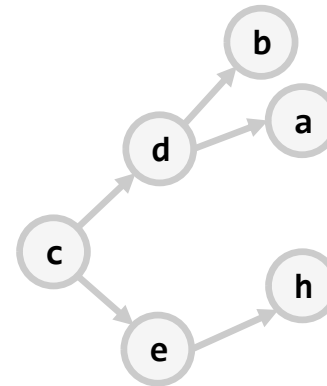
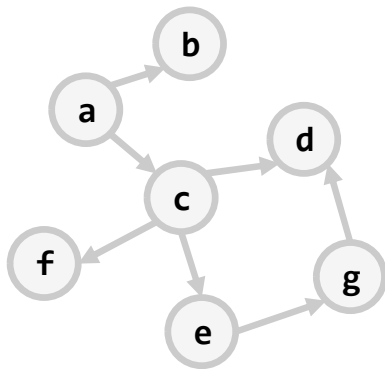
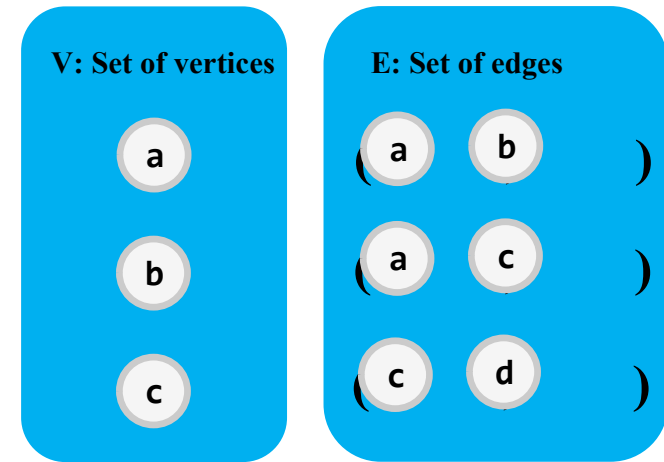
## □ Related topics

- Web Page Ranking
  - Vertices are web pages, edges are hyperlinks
- Wikipedia
  - Vertices are articles, edges are links



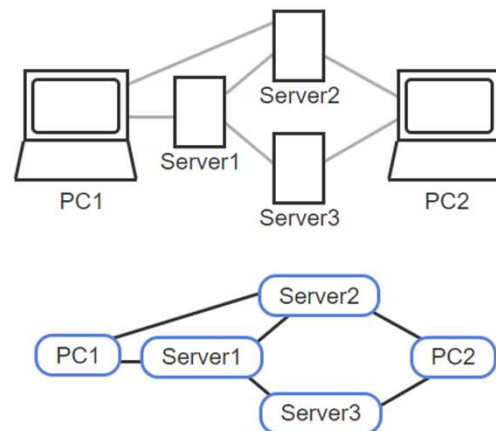
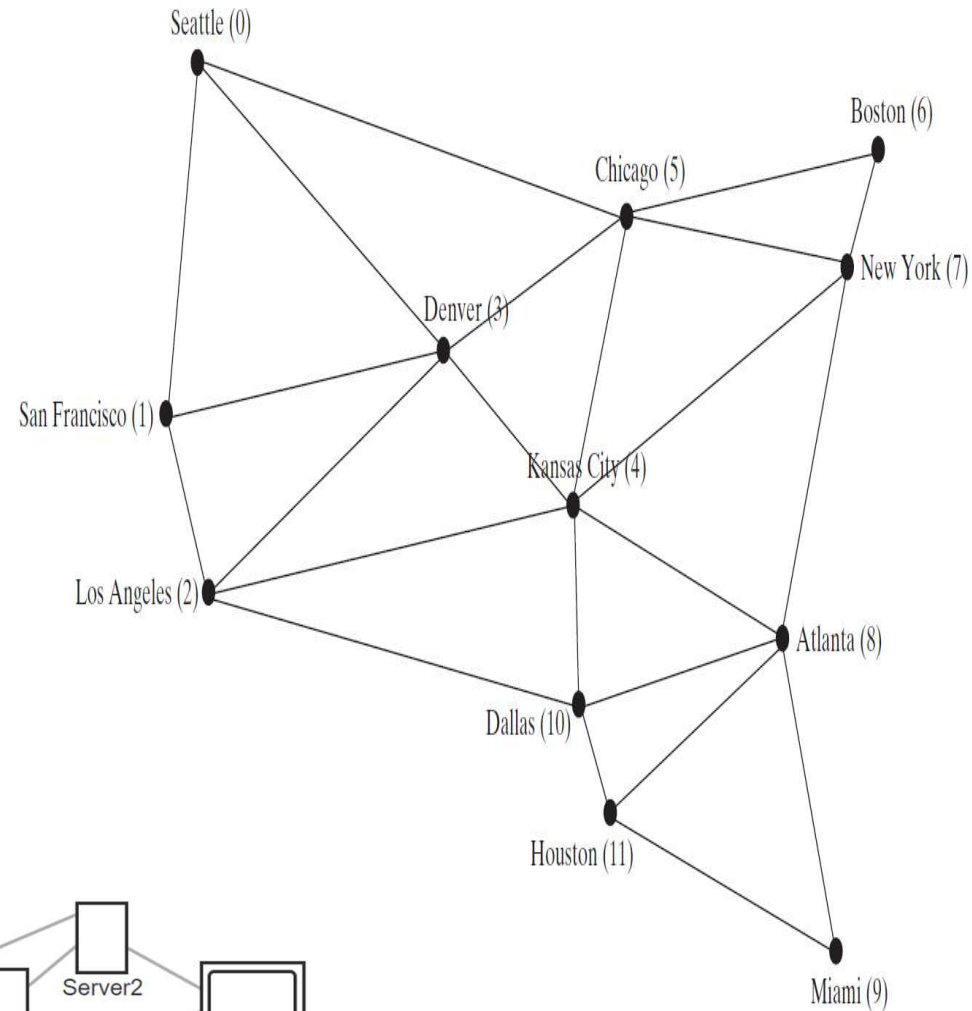
# Graphs

- A **Graph** consists of two sets,  $V$  and  $E$ :
  - $V$ : Set of **vertices** (aka **nodes**)
  - $E$ : Set of **edges** (pairs of vertices)
  - $|V|$ : Size of  $V$  (also called  $n$ )
  - $|E|$ : Size of  $E$  (also called  $m$ )

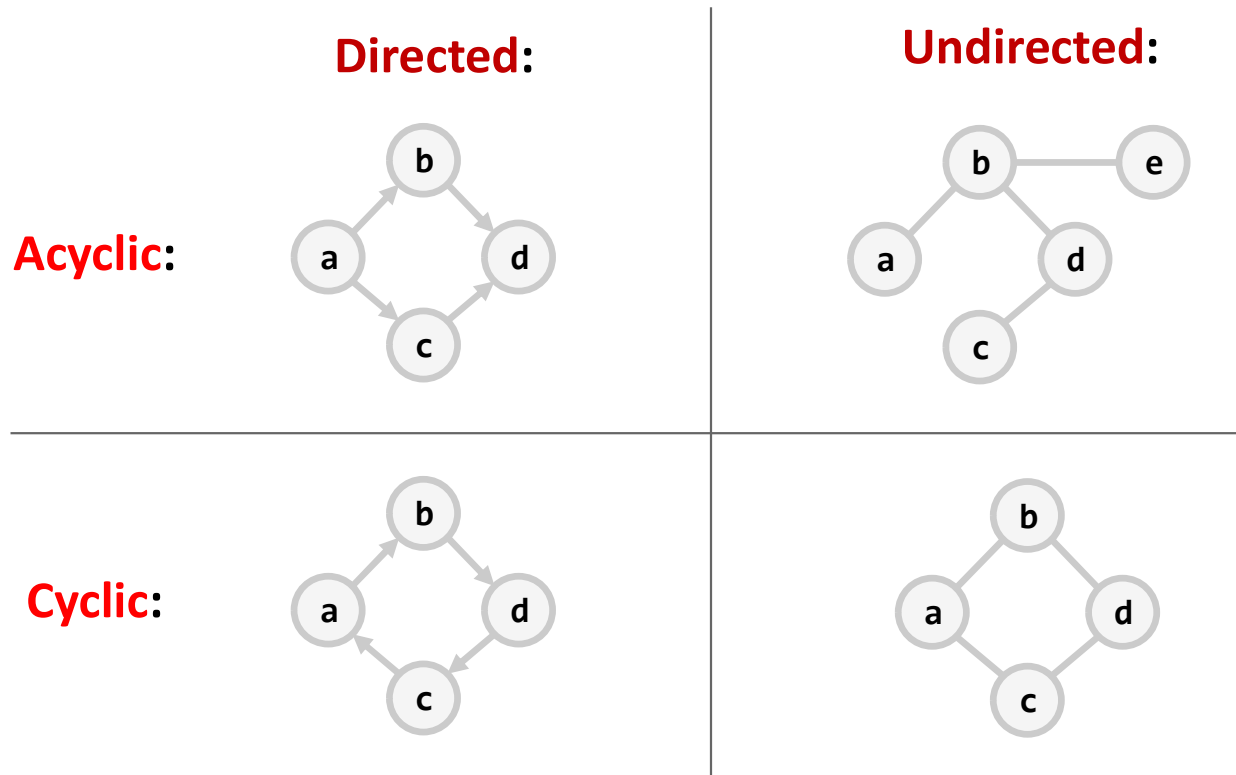


# Modeling Using Graphs

- In the given map
  - The cities(i) are vertices or nodes
  - The lines are called edges
- A subgraph is a portion of a graph that is itself a graph

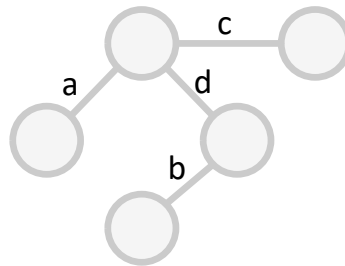
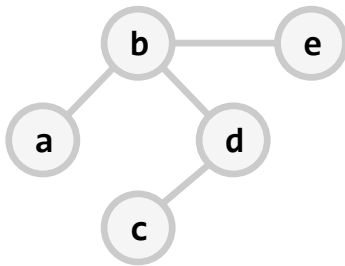


# Directed vs Undirected; Acyclic vs Cyclic

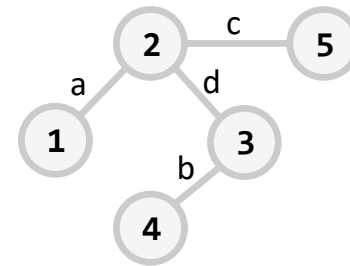


# Labeled and Weighted Graphs

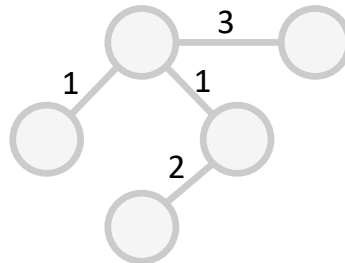
**Vertex Labels**



**Vertex & Edge Labels**



**Numeric Edge Labels  
(Edge Weights)**



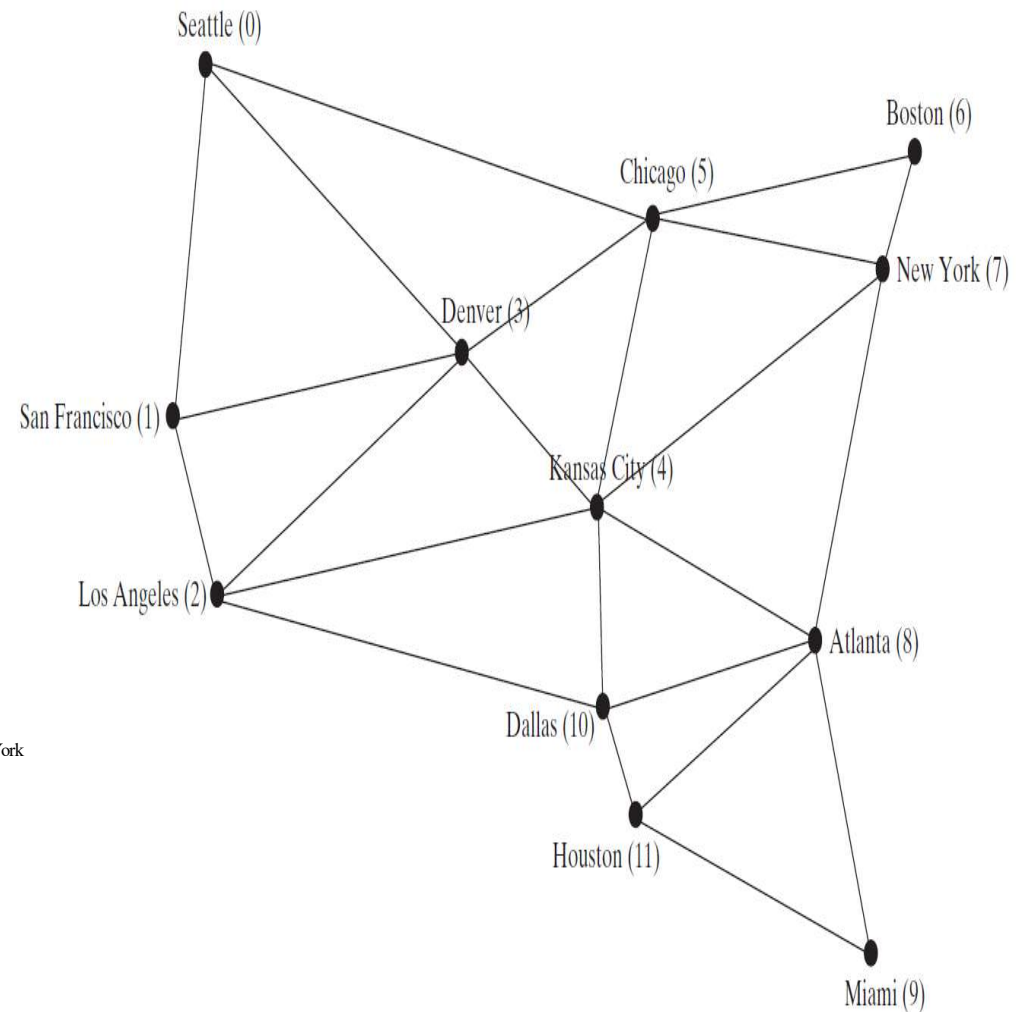
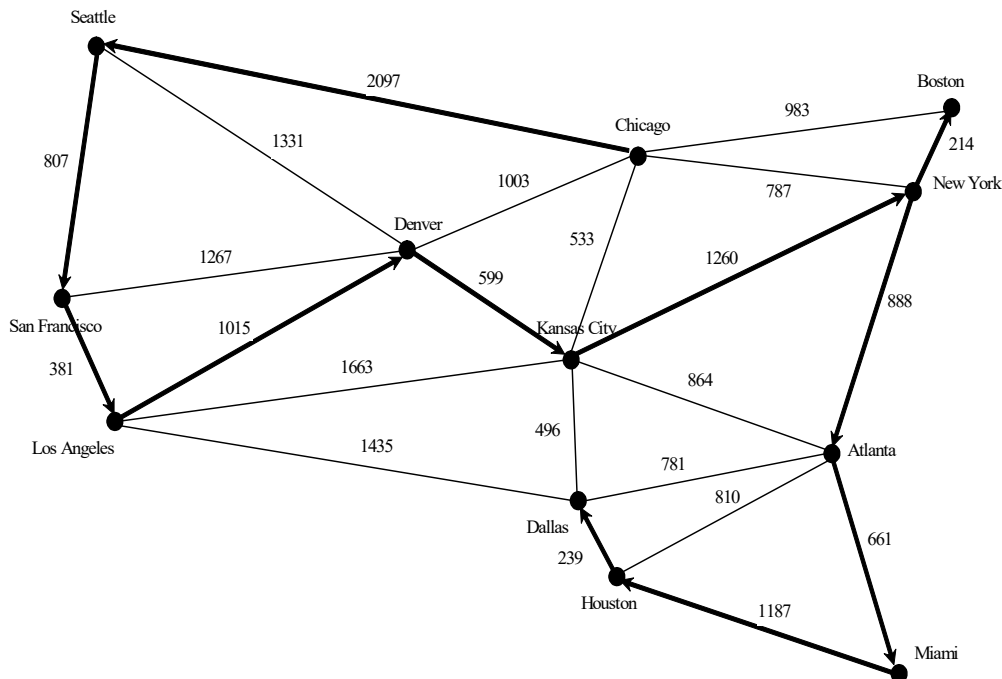
# Basic Graph Terminologies

What is a graph?  $G=(V, E)$

Define a graph

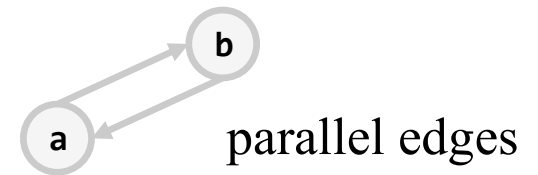
A path between two vertices

Weighted vs. unweighted graphs



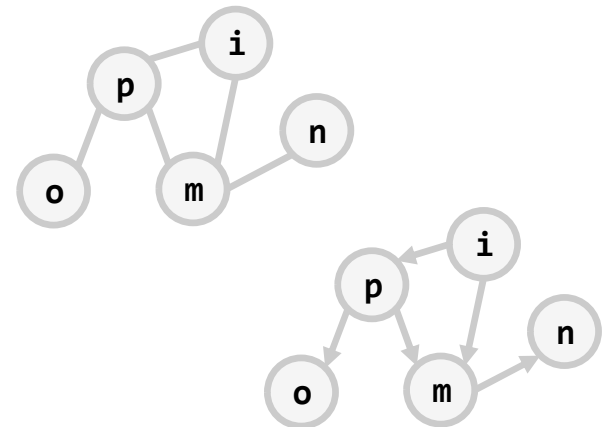
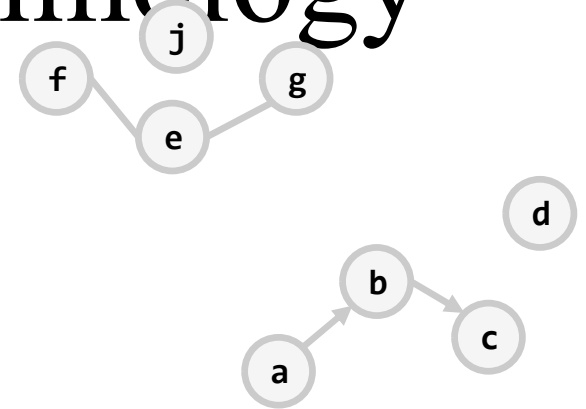
# More Graph Terminology

- A **Simple Graph** has no **self-loop** or **parallel edges**
  - Unless otherwise stated, all graphs in this course are simple
- Vertices with an edge between them are **adjacent**
  - Vertices or edges may have optional **labels**
    - Numeric edge labels are sometimes called **weights**



# More More Graph Terminology

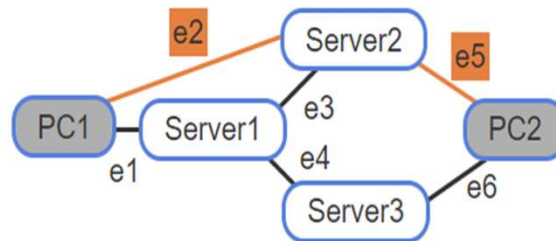
- Two vertices are **connected** if there is a path between them
  - If all the vertices are connected, we say the graph is **connected**
  - The number of edges leaving a vertex is its **degree**
  
- A **path** is a sequence of vertices connected by edges
  - A **simple path** is a path without repeated vertices
  - A **cycle** is a path whose first and last vertices are the same
    - A graph with a cycle is **cyclic**



# Basic Graph Terminologies

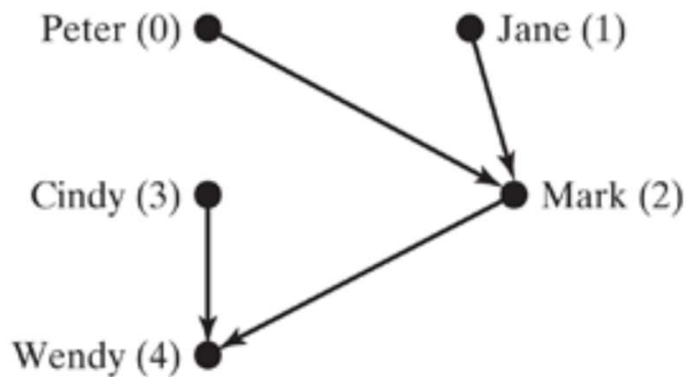
## □ Distance

- The number of edges on the shortest path between two vertices.

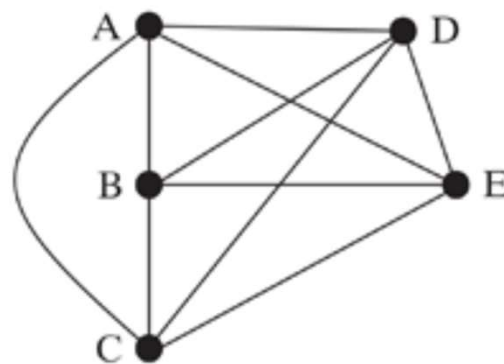


# Basic Graph Terminologies

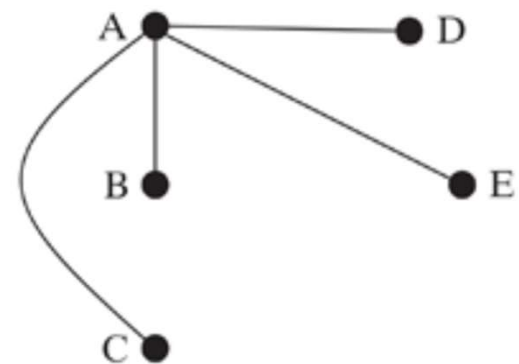
- Complete graph
  - Every two vertices are adjacent
- Subgraph of a graph  $G$ 
  - A graph whose vertex set is a subset of  $G$  and whose edge set is a subset of that of  $G$
- Neighbor
  - Two vertices are neighbors if they are adjacent



(a) A directed graph



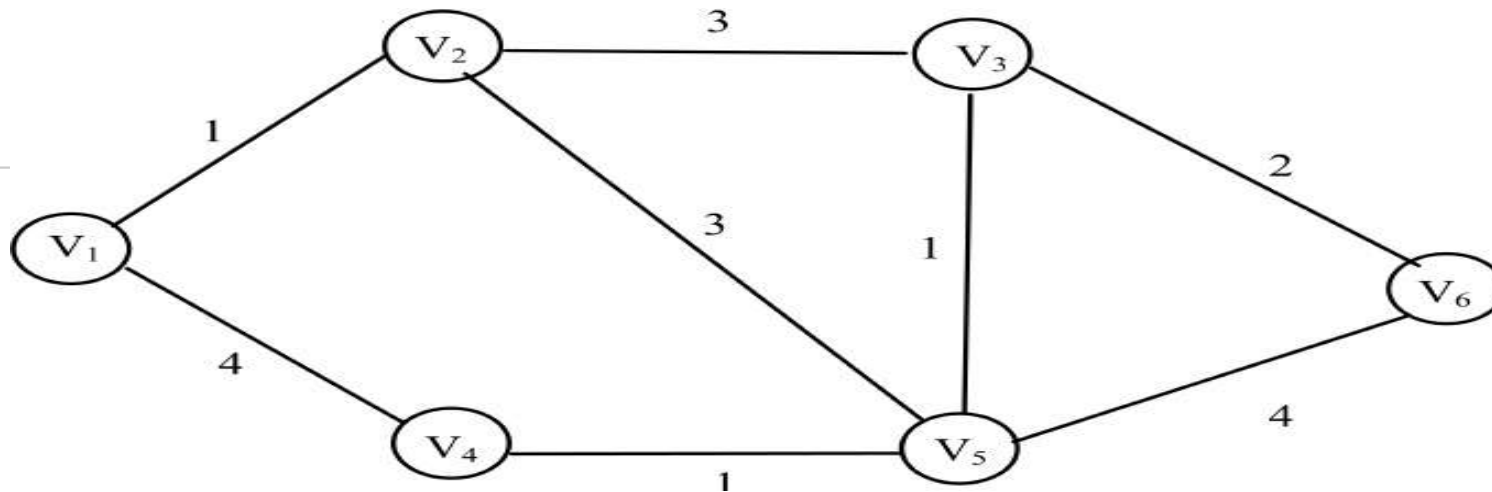
(b) A complete graph



(c) A subgraph of the graph in (b)

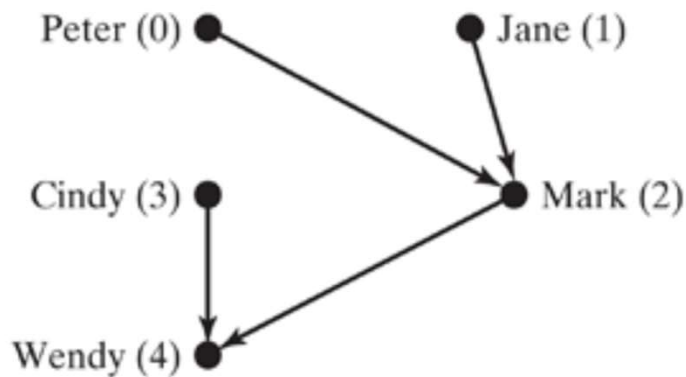
# Basic Graph Terminologies

- Simple path
  - A path that doesn't have any loops or parallel edges

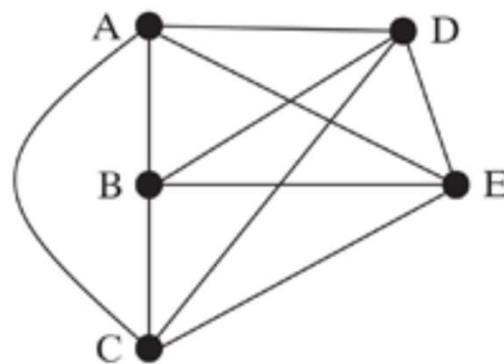


# Basic Graph Terminologies

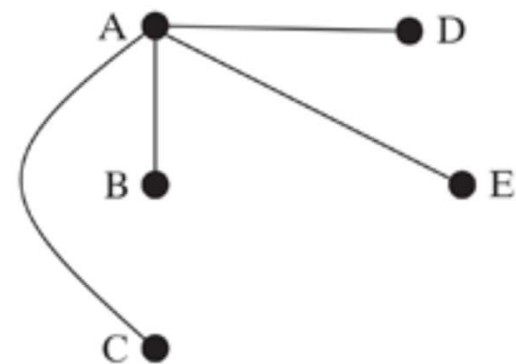
- Cycle
  - A closed path that starts from a vertex and ends at the same vertex
- Tree
  - A connected graph that does not have cycles
- Spanning tree of G
  - A connected subgraph of G, and the subgraph is a tree that contains all vertices in G



(a) A directed graph



(b) A complete graph



(c) A subgraph of the graph in (b)

# Some examples

- For each of the following: what should you choose for vertices and edges? Directed?
- **Webpages on the Internet**
  - **Vertices:** webpages. **Edges** from a to b if a has a hyperlink to b.
  - Directed, since hyperlinks go in one direction
- **Ways to walk between campus buildings**
  - **Vertices:** buildings. **Edges:** A street name or walkway that connects 2 buildings
  - Undirected, since each route can be walked both ways
- **Course Prerequisites**
  - **Vertices:** courses. **Edge:** from a to b if a is a prereq for b.
  - Directed, since one course comes before the other

# Lecture Outline

- Graphs
  - Definitions
  - Choosing Graph Types
- **Graph Implementations** ◀
- s-t Connectivity Problem

# Representing Graphs

Representing Vertices

Representing Edges: Edge Array

Representing Edges: Edge Objects

Representing Edges: Adjacency Matrices

Representing Edges: Adjacency Lists

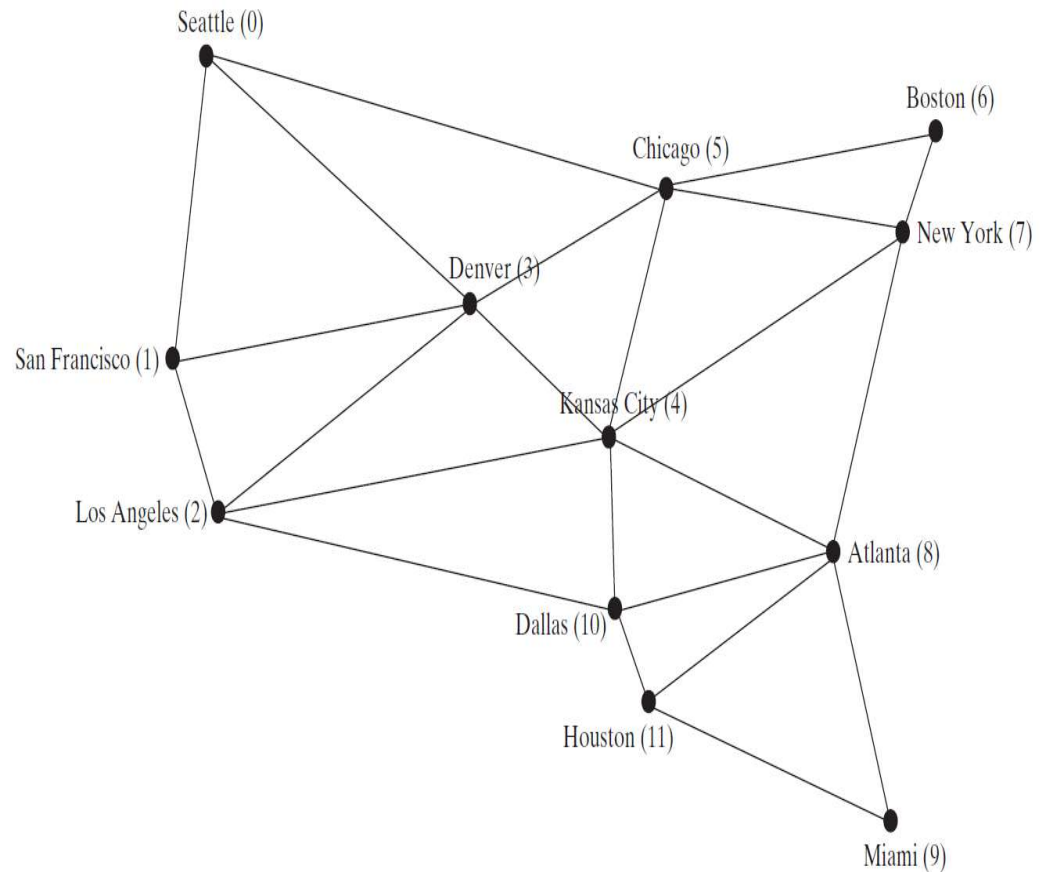
# Representing Vertices

```
String[] vertices = {"Seattle", "San Francisco", "Los Angeles",  
"Denver", "Kansas City", "Chicago", ... };
```

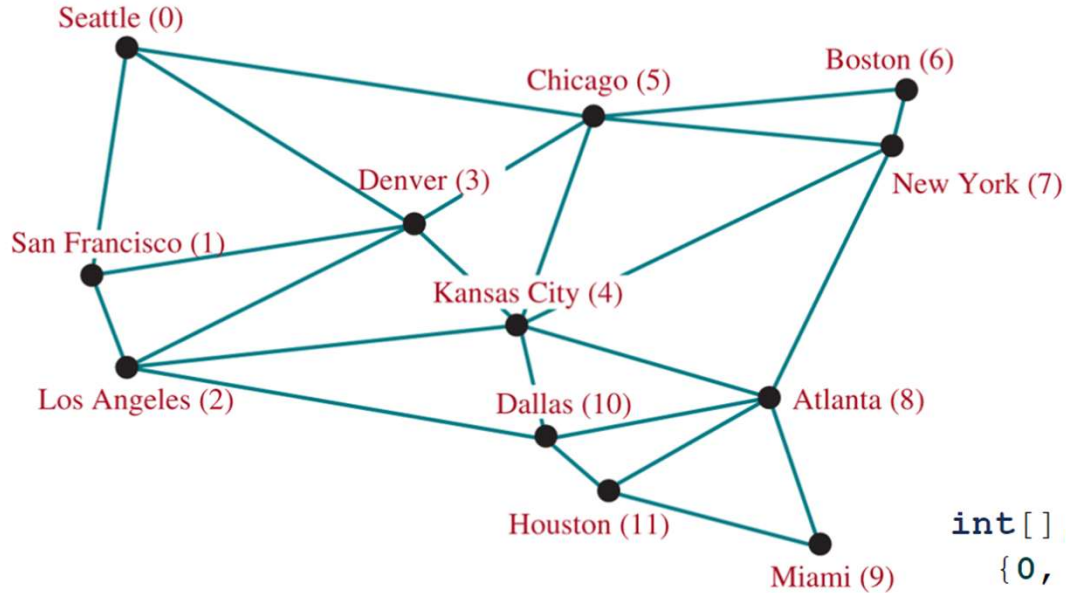
```
City[] vertices = {city0, city1, ... };
```

```
public class City {  
  
}
```

```
List<String> vertices;
```



# Representing Edges: Edge Array



```
int[][] edges = {  
    {0, 1}, {0, 3}, {0, 5},  
    {1, 0}, {1, 2}, {1, 3},  
    {2, 1}, {2, 3}, {2, 4}, {2, 10},  
    {3, 0}, {3, 1}, {3, 2}, {3, 4}, {3, 5},  
    {4, 2}, {4, 3}, {4, 5}, {4, 7}, {4, 8},  
    {4, 10},  
    {5, 0}, {5, 3}, {5, 4}, {5, 6}, {5, 7},  
    {6, 5}, {6, 7},  
    {7, 4}, {7, 5}, {7, 6}, {7, 8},  
    {8, 4}, {8, 7}, {8, 9}, {8, 10}, {8, 11},  
    {9, 8}, {9, 11},  
    {10, 2}, {10, 4}, {10, 8}, {10, 11},  
    {11, 8}, {11, 9}, {11, 10}  
};
```

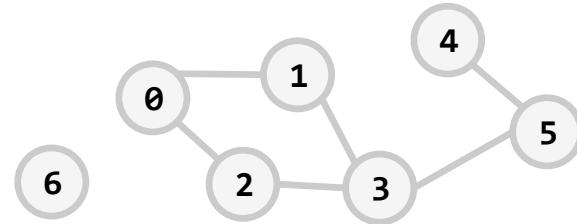
# Representing Edges: Edge Object

```
public class Edge {  
    int u, v;  
    public Edge(int u, int v) {  
        this.u = u;  
        this.v = v;  
    }  
}
```

```
List<Edge> list = new ArrayList<>();
```

```
list.add(new Edge(0, 1)); list.add(new Edge(0, 3)); ...
```

# Adjacency Matrix

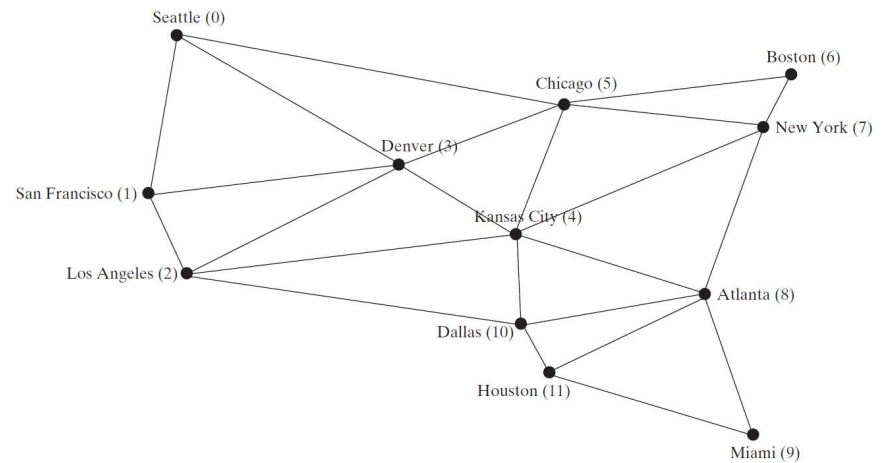


- Create a 2D matrix that is  $|V| \times |V|$
- In an adjacency matrix,  $a[u][v]$  is 1 if there is an edge  $(u,v)$ , and 0 otherwise.
- Symmetric for undirected graphs

|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 1 | 0 | 0 | 0 |
| 2 | 1 | 0 | 0 | 1 | 0 | 0 | 0 |
| 3 | 0 | 1 | 1 | 0 | 0 | 1 | 0 |
| 4 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 5 | 0 | 0 | 0 | 1 | 1 | 0 | 0 |
| 6 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

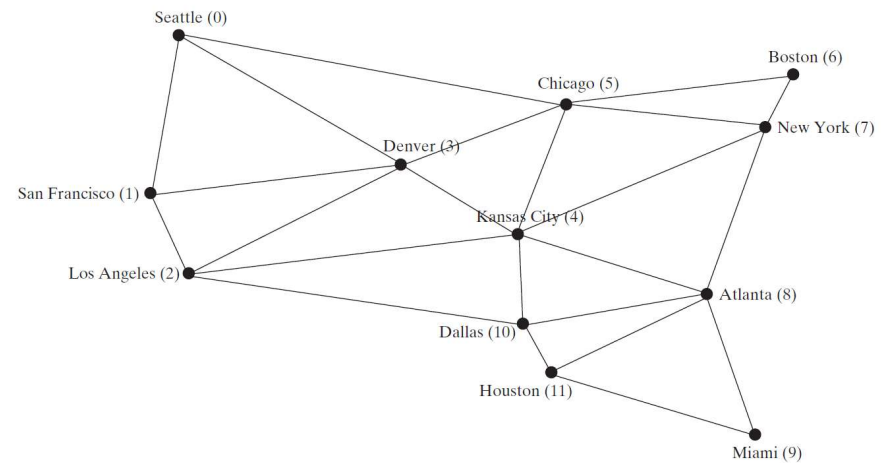
# Representing Edges: Adjacency Matrix

```
int[][] adjacencyMatrix = {
    {0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0}, // Seattle
    {1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0}, // SF
    {0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0}, // LA
    {1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0}, // Denver
    {0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0}, // Kansas
    {1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0}, // Chicago
    {0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0}, // Boston
    {0, 0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0}, // NY
    {0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1}, // Atlanta
    {0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1}, // Miami
    {0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1}, // Dallas
    {0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0} // Houston
};
```



# Representing Edges: Adjacency Vertex List

|               |               |   |    |    |    |    |    |
|---------------|---------------|---|----|----|----|----|----|
| Seattle       | neighbors[0]  | 1 | 3  | 5  |    |    |    |
| San Francisco | neighbors[1]  | 0 | 2  | 3  |    |    |    |
| Los Angeles   | neighbors[2]  | 1 | 3  | 4  | 10 |    |    |
| Denver        | neighbors[3]  | 0 | 1  | 2  | 4  | 5  |    |
| Kansas City   | neighbors[4]  | 2 | 3  | 5  | 7  | 8  | 10 |
| Chicago       | neighbors[5]  | 0 | 3  | 4  | 6  | 7  |    |
| Boston        | neighbors[6]  | 5 | 7  |    |    |    |    |
| New York      | neighbors[7]  | 4 | 5  | 6  | 8  |    |    |
| Atlanta       | neighbors[8]  | 4 | 7  | 9  | 10 | 11 |    |
| Miami         | neighbors[9]  | 8 | 11 |    |    |    |    |
| Dallas        | neighbors[10] | 2 | 4  | 8  | 11 |    |    |
| Houston       | neighbors[11] | 8 | 9  | 10 |    |    |    |



# Representing Edges: Adjacency Edge List

|               |               |             |             |              |              |             |             |  |  |  |  |  |
|---------------|---------------|-------------|-------------|--------------|--------------|-------------|-------------|--|--|--|--|--|
| Seattle       | neighbors[0]  | Edge(0, 1)  | Edge(0, 3)  | Edge(0, 5)   |              |             |             |  |  |  |  |  |
| San Francisco | neighbors[1]  | Edge(1, 0)  | Edge(1, 2)  | Edge(1, 3)   |              |             |             |  |  |  |  |  |
| Los Angeles   | neighbors[2]  | Edge(2, 1)  | Edge(2, 3)  | Edge(2, 4)   | Edge(2, 10)  |             |             |  |  |  |  |  |
| Denver        | neighbors[3]  | Edge(3, 0)  | Edge(3, 1)  | Edge(3, 2)   | Edge(3, 4)   | Edge(3, 5)  |             |  |  |  |  |  |
| Kansas City   | neighbors[4]  | Edge(4, 2)  | Edge(4, 3)  | Edge(4, 5)   | Edge(4, 7)   | Edge(4, 8)  | Edge(4, 10) |  |  |  |  |  |
| Chicago       | neighbors[5]  | Edge(5, 0)  | Edge(5, 3)  | Edge(5, 4)   | Edge(5, 6)   | Edge(5, 7)  |             |  |  |  |  |  |
| Boston        | neighbors[6]  | Edge(6, 5)  | Edge(6, 7)  |              |              |             |             |  |  |  |  |  |
| New York      | neighbors[7]  | Edge(7, 4)  | Edge(7, 5)  | Edge(7, 6)   | Edge(7, 8)   |             |             |  |  |  |  |  |
| Atlanta       | neighbors[8]  | Edge(8, 4)  | Edge(8, 7)  | Edge(8, 9)   | Edge(8, 10)  | Edge(8, 11) |             |  |  |  |  |  |
| Miami         | neighbors[9]  | Edge(9, 8)  | Edge(9, 11) |              |              |             |             |  |  |  |  |  |
| Dallas        | neighbors[10] | Edge(10, 2) | Edge(10, 4) | Edge(10, 8)  | Edge(10, 11) |             |             |  |  |  |  |  |
| Houston       | neighbors[11] | Edge(11, 8) | Edge(11, 9) | Edge(11, 10) |              |             |             |  |  |  |  |  |

# Representing Adjacency Edge List Using ArrayList

```
List<ArrayList<Edge>> neighbors = new ArrayList<>();  
neighbors.add(new ArrayList<Edge>());  
neighbors.get(0).add(new Edge(0, 1));  
neighbors.get(0).add(new Edge(0, 3));  
neighbors.get(0).add(new Edge(0, 5));  
neighbors.add(new ArrayList<Edge>());  
neighbors.get(1).add(new Edge(1, 0));  
neighbors.get(1).add(new Edge(1, 2));  
neighbors.get(1).add(new Edge(1, 3));  
...  
...  
neighbors.get(11).add(new Edge(11, 8));  
neighbors.get(11).add(new Edge(11, 9));  
neighbors.get(11).add(new Edge(11, 10));
```